Multi-Vehicle Chassis Control

Multi-Vehicle Chassis Control

1. Course Content
2. Preparation
3. Case Demonstration
4. Multi-Device Communication and Node Topology Verification
5. View the Multi-Device Computation Graph Topology
6. Core Source Code Analysis

1. Course Content

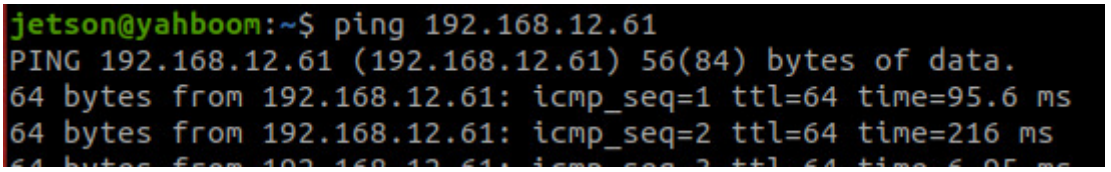
- Through gamepad or keyboard control, achieve synchronous control of multiple robot vehicle chassis.
- Under the same local area network and the same `ROS_DOMAIN_ID`, the primary vehicle, follower vehicles, and the control terminal (virtual machine/PC) can achieve shared control or grouped control through topic remapping or independent namespaces. This section uses two vehicles (`robot1` and `robot2`) as an example to introduce two modes: shared keyboard control and independent gamepad distribution control.

---

2. Preparation

- Taking two vehicles as an example, the following requirements must be met simultaneously:

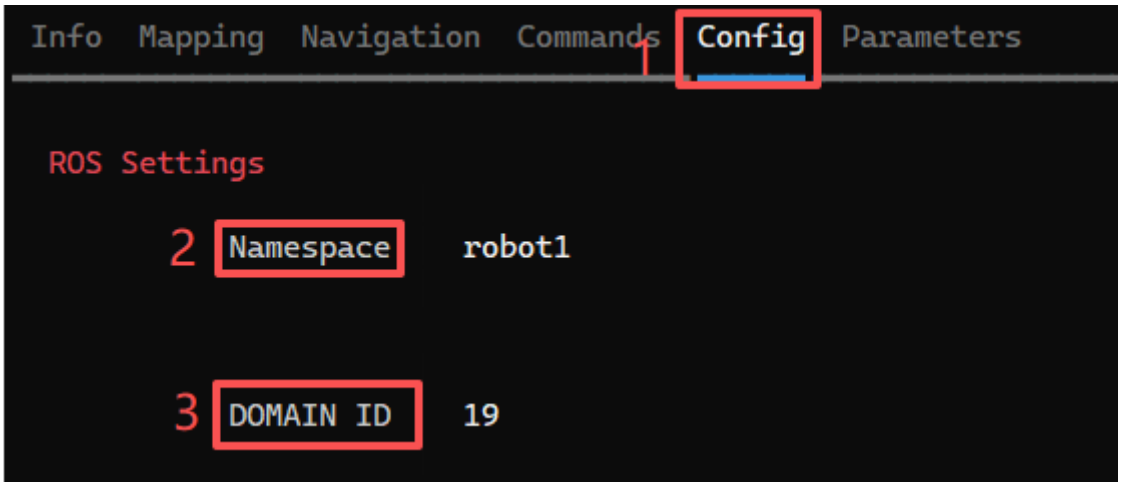
Same local area network: Connect to the same 2.4G/5G Wi-Fi router, ensuring the IPs are on the same subnet and can ping each other;

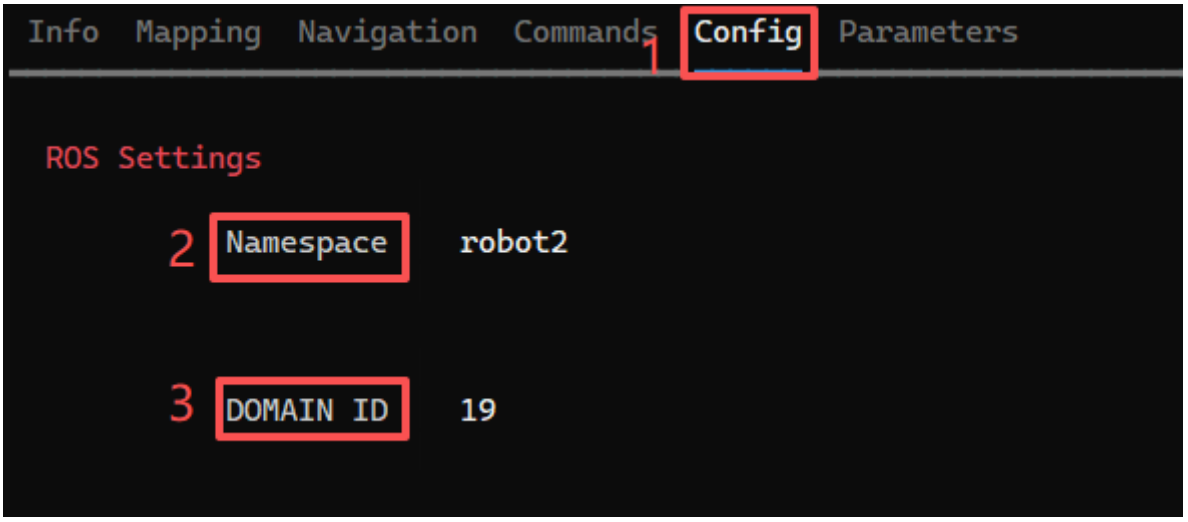


Same Domain ID: The `ROS_DOMAIN_ID` of both vehicles must be configured the same (the system default is usually 19, which usually does not need to be modified), **set different namespaces**

```
robot_manage gui
```

- After the modification is complete, click save for the configuration to take effect.





- Start the control program on each of the two vehicles; one vehicle is the primary vehicle and the others are follower vehicles.
- Primary vehicle (robot1) start

```
ros2 launch yahboomcar_bringup multi_robot_demo.launch.py demo:=control
```

- All other follower vehicles start

```
ros2 launch yahboomcar_bringup multi_robot_demo.launch.py
```

3. Case Demonstration

- Synchronously control the two vehicles via gamepad or keyboard. The demonstration case here uses keyboard control as an example. Start keyboard control in the terminal of any vehicle.

```
ros2 run common_demo keyboard_control
```

- At this point, you can synchronously control the robotic arms and chassis of both vehicles via the keyboard. For keyboard operations, refer to 【Keyboard Control】.

4. Multi-Device Communication and Node Topology Verification

```
ros2 topic list
```

You can see topics containing the dedicated namespaces of both vehicles (as well as global shared topics in keyboard mode):

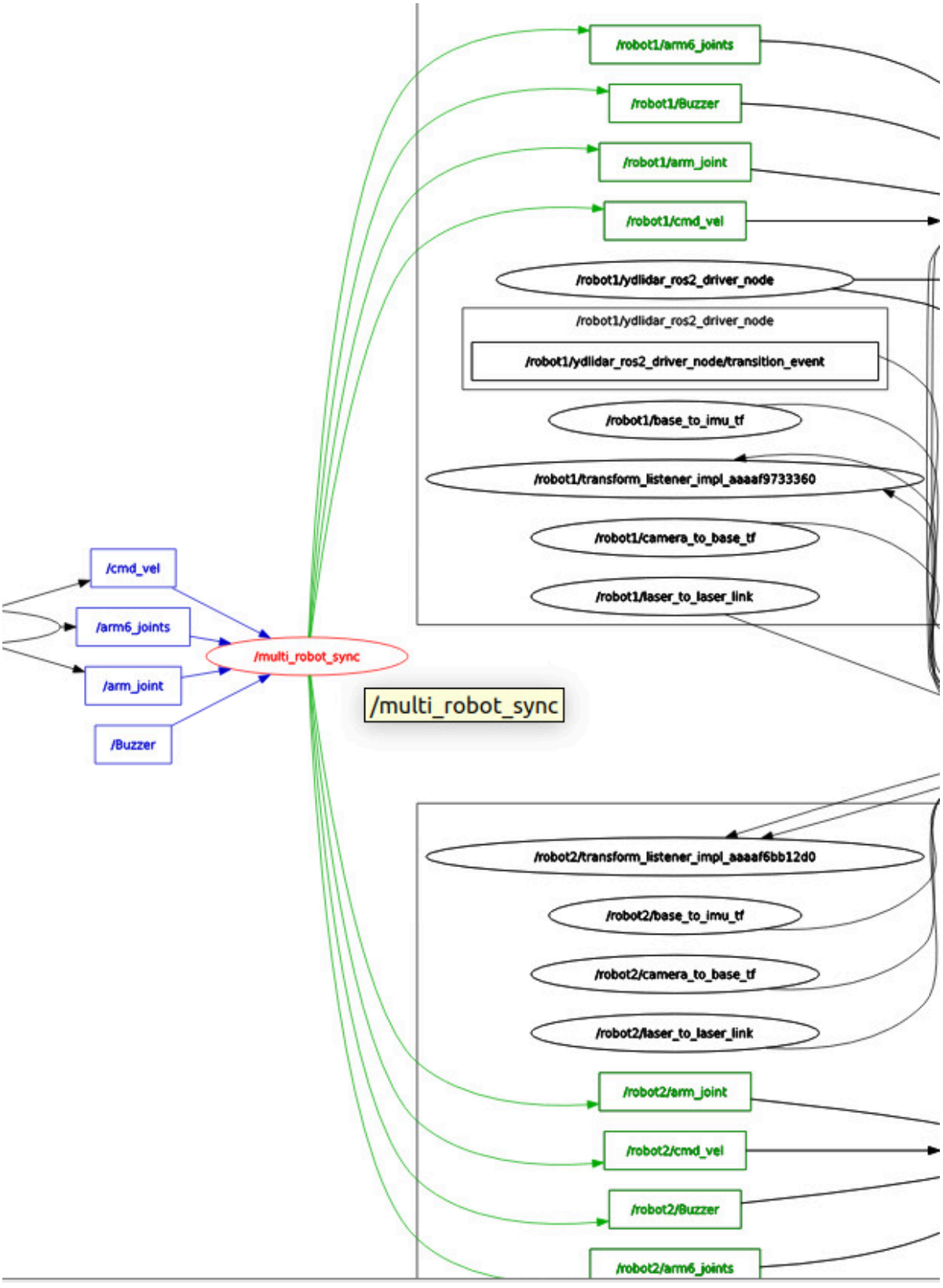
```
/robot1/Buzzer
/robot1/arm6_joints
/robot1/arm_joint
/robot1/arm_status/arm_joint
/robot1/cmd_vel
/robot1/imu/data_raw
/robot1/joint_states
/robot1/odom
/robot1/odom_raw
/robot1/point_cloud
/robot1/robot_description
/robot1/scan
/robot1/set_pose
/robot1/ydlidar_ros2_driver_node/transition_event
/robot2/Buzzer
/robot2/arm6_joints
/robot2/arm_joint
/robot2/arm_status/arm_joint
/robot2/cmd_vel
/robot2/imu/data_raw
/robot2/joint_states
/robot2/odom
/robot2/odom_raw
/robot2/point_cloud
/robot2/robot_description
/robot2/scan
/robot2/set_pose
/robot2/ydlidar_ros2_driver_node/transition_event
```

5. View the Multi-Device Computation Graph Topology

Run the rqt computation graph analysis tool in the terminal:

```
rqt_graph
```

The computation graph visually shows the communication paths between each control node and the vehicle driver nodes. The multi_robot_sync node subscribes to the gamepad or keyboard control topics and distributes the data to robot1 and robot2.



6. Core Source Code Analysis

- Launch path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/multi_robot/multi_
robot_sync.py
```

SYNC_TOPICS dictionary description:

- `SYNC_TOPICS`: Defines the topics that need to be synchronously forwarded and their corresponding message types; the node creates a subscription and a publisher for each topic in this dictionary.
- `'cmd_vel': Twist`: The chassis velocity command topic, whose message type is the geometry velocity message `Twist` (containing linear velocity and angular velocity).
- `'Buzzer': Bool`: The buzzer control topic, whose message type is the boolean value `Bool`.
- `'arm6_joints': ArmJoints`: The 6-joint control command topic, whose message type is the custom `ArmJoints` (containing 6 joint angles and the motion time).
- `'arm_joint': ArmJoint`: The gripper joint command topic, whose message type is the custom `ArmJoint` (a single joint angle).

MultiRobotSync class constructor description:

- `Node('multi_robot_sync')`: The node name is `multi_robot_sync`.
- `declare_parameter('robot_names', ['robot1', 'robot2'])`: Declares the robot name list parameter, defaulting to `['robot1', 'robot2']`, which can be overridden via the `-p` command line.
- `name.strip().strip('/')`: Removes the whitespace and slashes at both ends of the name to prevent namespace format errors.
- `self.publishers_map`: A dictionary structure, where the key is the topic name and the value is the list of publishers for each robot corresponding to that topic.
- `create_publisher(msg_type, f'/{name}/{topic}', 10)`: Creates a publisher for each robot under the corresponding namespace, such as `/robot1/cmd_vel` and `/robot2/cmd_vel`.
- `create_subscription(msg_type, topic, ..., 10)`: Subscribes to the global (non-namespaced) topic to receive control commands issued by the keyboard or gamepad.
- `lambda msg, t=topic: self.sync_callback(t, msg)`: A closure callback function; `t=topic` binds the current topic name into the callback, ensuring that callbacks of different topics can be correctly distinguished.

sync_callback function description:

- `sync_callback(self, topic, msg)`: Receives the topic name and the message object, and forwards the message to the corresponding topic publishers of all robots.
- `self.publishers_map[topic]`: Gets the list of publishers for all robots under that topic according to the topic name.

- `pub.publish(msg)`: Publishes the same message to each robot one by one, achieving multi-vehicle synchronous control.

```
SYNC_TOPICS = {
    'cmd_vel': Twist,
    'Buzzer': Bool,
    'arm6_joints': ArmJoints,
    'arm_joint': ArmJoint,
}


class MultiRobotSync(Node):
    def __init__(self):
        super().__init__('multi_robot_sync')

        self.declare_parameter('robot_names', ['robot1', 'robot2'])
        robot_names = [name.strip().strip('/') for name in
                       self.get_parameter('robot_names').value if name.strip()]
        if not robot_names:
            self.get_logger().error('robot_names is empty, nothing to sync')


        self.publishers_map = {}
        for topic, msg_type in SYNC_TOPICS.items():
            self.publishers_map[topic] = [
                self.create_publisher(msg_type, f'/{name}/{topic}', 10)
                for name in robot_names
            ]
            self.create_subscription(
                msg_type, topic,
                lambda msg, t=topic: self.sync_callback(t, msg), 10)

        self.get_logger().info(
            f'multi_robot_sync started, robots: {robot_names}, '
            f'topics: {list(SYNC_TOPICS.keys())}')

    def sync_callback(self, topic, msg):
        for pub in self.publishers_map[topic]:
            pub.publish(msg)
```